

Impact and Trends in HNSW Insertion Order - Research Report

Sovannratana Khek

Department of Computer Science — CS 546 - Parallel Programming

Illinois Institute of Technology

skhek@hawk.illinoistech.edu

Abstract—Hierarchical Navigable Small World (HNSW) graphs are the dominant index structure for approximate nearest neighbor search in modern vector databases and retrieval-augmented generation systems. While HNSW construction parameters such as `ef_construction` and `M` are well studied, the effect of vector insertion ordering on graph quality remains underexplored in the context of text embedding datasets. This paper investigates five insertion strategies — random ordering, K-Means clustering, Hilbert curve ordering, Local Intrinsic Dimensionality (LID) ordering, and a K-Means density approximation — evaluated across two BEIR benchmark datasets of differing scale and two `ef_construction` configurations. Graph quality was measured via `Recall@10` against an exact brute force index, with `NDCG@10`, build time, preprocessing time, and retrieval time recorded as secondary metrics. Results show that random ordering consistently outperformed most structured alternatives across all conditions, with the recall gap widening under larger corpus size and lower `ef_construction` values. The K-Means density approximation matched random ordering on `Recall@10` while providing a structured, geometrically motivated alternative at a fraction of the cost of exact LID computation. `NDCG@10` and retrieval time showed no meaningful differences across strategies. These findings are interpreted through the framework of Local Intrinsic Dimensionality, where structured orderings inadvertently delay the insertion of high LID vectors that are critical for establishing global graph connectivity. The results suggest that at moderate corpus scales, the default random insertion order remains competitive, though the density approximation strategy offers a promising structured alternative warranting further investigation at larger scales.

Index Terms—HNSW, vector databases, approximate nearest neighbor search, insertion ordering, retrieval augmented generation, local intrinsic dimensionality

I. INTRODUCTION

Hierarchical Navigable Small World (HNSW) is, simply put, a nearest neighbor search algorithm. The basic workings of the algorithm functions by building a graph of nodes using a specific hierarchical or layered structure. A search is performed on the graph via an entry layer and a query. The traversal searches greedily to find the closest node in that layer. Once found, it goes to the next layer, repeating the process until the final layer, where it returns the node, an approximate nearest neighbor to the query. The core idea of HNSW is the layered system; higher initial layers act as long distance jumps across the node space, and as you go down the layers, the hops shift to smaller adjustments in a more local search. This optimizes the search by their hops; you make large adjustments until you are close and then fine tune and narrow your search,

all done just by the structure of the graph rather than an exhaustive or brute force search which are taxing in large spaces. HNSW is widely adopted in vector database systems underpinning modern retrieval-augmented generation (RAG) pipelines. Representing information in the form of a node, building a graph from it, and being able to efficiently query into said graph has many applications in the data industry.

This research project focuses on one step in the graph building step of HNSW, insertion. Inserting nodes is done one at a time; the modern default order of insertion is randomized. Due to how the edges are built, early nodes are established as central hubs for traversal as they create more diverse connections to preceding nodes. This can affect the accuracy and efficiency of the search, which are important metrics when these graphs are used in RAG systems or any other downstream application. The project aims to test four alternative methods of insertion — K-Means clustering, Hilbert curve ordering, LID-based ordering, and a K-Means density approximation — each aiming to find good initial vector insertions to act as those central hubs. The K-Means clustering approach was expected to outperform randomization as the method aims to find k points in the vector space that effectively centralize a group of nodes and were inserted first so that those central hubs would be spread across the space rather than randomized. After experimenting on two datasets with two different sizes and `ef_construction` factors, randomization and the density approximation stood out as the strongest approaches for query accuracy when checking against the brute force search.

II. RELATED WORK

HNSW was originally proposed by Malkov and Yashunin [1] as a graph-based approximate nearest neighbor search algorithm. They established the randomized layering and quality control through an `ef_construction`, which controls the size of the candidate pool considered when forming edges during graph construction. This project builds on the standard HNSW construction algorithm, investigating how pre-determined insertion orderings affect the resulting graph quality.

Elliott et al. [2] demonstrated that insertion order can shift recall by up to 12 percentage points. The approach used was to identify nodes that were hard to reach by their relative distance to neighbors. To do this, they checked each node and found the rate at which the density of neighbors grows with distance,

reflecting the local geometric complexity around each point — a metric labeled Local Intrinsic Dimensionality (LID). Higher LID nodes were inserted first so they were more accessible while the low LID nodes were inserted last since they were highly local nodes that could be found during the fine-tuning, lower-layered search phase. Their work spans seventeen embedding models across seven datasets ranging from 3.6K to 547K documents, using a fixed `ef_construction` of 128 derived from a survey of production HNSW systems. This work helps explain the metrics found in this project, identifying clear factors that affect a HNSW graph’s effectiveness.

For evaluation, we use the BEIR benchmark [3], a heterogeneous collection of information retrieval datasets with precomputed relevance judgments.

III. METHODOLOGY

A. Dataset

SciFact and FiQA are datasets from BEIR, each differing in size as corpus scale may affect certain metrics. SciFact is themed in the sciences with ~5,000 documents while FiQA is themed in finance with ~57,000 documents. BEIR provides built-in benchmarking utilities packaged together with the dataset. A fetch to their API returns a zip file that can be loaded and used while their utility functions are available in their Python package.

B. Embedding Model

To work with HNSW, the documents need to be formatted into vectors to act as nodes. Embedding the documents into vectors was done with all-MiniLM-L6-v2, a sentence transformer model with 384-dimensional output, for its general purpose design and it being standard for BEIR experiments. Embeddings were precomputed and cached to disk before any insertion experiments to ensure all strategies operated on identical vector representations.

C. Index Configuration

The experiment only varied by insertion strategy. For the HNSW graph parameters, all configurations were kept constant between trials, tests, and datasets:

TABLE I
FIXED HNSW PARAMETERS

Parameter	Value
M	16
<code>ef_construction</code>	200, 32
<code>ef_search</code>	50
Space	Cosine
Top-K	10

Two `ef_construction` values were tested to examine whether the candidate pool size during construction masks or amplifies the effect of insertion ordering. All other parameters were held constant to isolate insertion order as the independent variable.

D. Insertion Strategies

Randomizing the insertion order of vectors required establishing the list of candidates and shuffling the order each time we build the HNSW graph. This acted as the baseline since this is the default method for modern HNSW implementations. Random ordering ensures no systematic bias is introduced during construction, serving as a neutral baseline against which structured orderings can be compared.

K-Means Clustering involved determining what number of clusters best fit all nodes for the dataset. The number of clusters k was set to the square root of the corpus size, a common heuristic. Each cluster would be represented by a centroid vector that centralized related nodes into its cluster. A k -means calculation was performed to find each centroid vector from the whole vector space. Each vector in the corpus is assigned to its nearest centroid, grouping the corpus into k clusters. The closest corpus vector was used to represent the centroid so the abstract computed value had meaning as a real document. The insertion starts with all the centroid representatives, followed by each cluster’s member set. The idea of this approach was to first establish central hubs for related or close vectors before inserting the members of each grouping.

Hilbert Curve fitting was used to determine an insertion order that preserves locality between nodes. Close neighbors would be sorted to be next to each other using the curve. Since the Hilbert curve library requires integer coordinates, vectors were quantized from 32-bit floats to 16-bit unsigned integers via min-max normalization before curve position computation. This approach aimed to improve graph connectivity by ensuring spatially proximate vectors were inserted consecutively. The hypothesis was that when a vector is inserted, its nearest neighbors would already be present in the graph from the previous insertions, allowing HNSW to form more accurate edges during construction.

LID-based ordering was implemented as a direct application of the approach described by Elliott et al. [2]. For each corpus vector, the 100 nearest neighbors were retrieved using an exact brute force index, matching the neighbor count used in their work. The Maximum Likelihood Estimator (MLE) for LID was then applied:

$$\widehat{LID}(x) = - \left(\frac{1}{k} \sum_{i=1}^k \log \frac{r_i}{r_{\max}} \right)^{-1} \quad (1)$$

where r_i is the distance to the i -th nearest neighbor and $r_{\max}$ is the distance to the k -th neighbor. Vectors were then sorted in descending order of LID score, inserting the highest LID vectors first. This is an $O(n^2)$ operation and represents the primary computational cost of the strategy.

It is important to note several differences between this implementation and that of Elliott et al. [2]. Their study evaluated seventeen embedding models across seven datasets ranging from 3.6K to 547K documents, providing significantly more statistical coverage. Their fixed `ef_construction` of 128 differs from the values tested here (200 and 32). Additionally,

their SciFact corpus contained $\sim 25,000$ documents compared to the $\sim 5,000$ document split used in this study, which may produce less stable LID estimates at the smaller scale. These differences should be considered when comparing results against their reported findings. The LID implementation in this study is a naive application of the MLE formula and has not been optimized to the degree that may be present in their released embeddings and estimates.

The K-Means Density Approximation is an original iteration of the LID concept proposed in this study as a computationally tractable alternative. Rather than computing exact LID via $O(n^2)$ brute force search, this method approximates outlieriness using the k-means cluster structure already available from the K-Means strategy. For each vector, the minimum cosine distance to any cluster centroid is computed across all k centroids. Vectors that are far from every centroid do not belong cleanly to any semantic neighborhood and are identified as outliers — approximating high LID behavior. Vectors are then sorted in descending order of minimum centroid distance and inserted in that order, placing the most outlier-like vectors first. This avoids the full $O(n^2)$ cost of exact LID computation while preserving the core geometric intuition: inserting geometrically isolated vectors first to establish diverse, long-range connections in the graph.

E. Evaluation Metrics

A different random seed was used across the 10 trials performed per strategy to ensure variance in random orderings was captured. The embedded vector set was cached to disk so that it stayed constant per dataset, only needing to be loaded into memory before each test. Our project collected the following metrics per test:

- **Build Time:** The average time to build the HNSW graph.
- **Recall@10:** The proportion of true nearest neighbors found by exact brute force search that are also returned by HNSW, measuring graph approximation quality.
- **NDCG@10:** A ranking quality metric computed against BEIR’s precomputed relevance judgments, measuring downstream retrieval usefulness.
- **Preprocessing Time:** The average time cost to precompute insertion ordering, applicable to all strategies except random.
- **Total Time:** Combined preprocessing and build time, representing the full cost of each strategy.
- **Average Retrieval Time:** The average time to retrieve results for each query in the BEIR query set.

IV. RESULTS

Across all four experimental conditions, the ranking of strategies by Recall@10 remained largely consistent. Random ordering and the density approximation led across most conditions, followed by K-Means, LID, and Hilbert. Notably, the density approximation matched random ordering exactly on SciFact at both `ef_construction` values — the first structured strategy to do so.

TABLE II
SciFACT RESULTS, `EF_CONSTRUCTION=200`

Strategy	Build (s)	Recall@10	NDCG@10	Retrieval (ms)
Random	0.9451	1.0000	0.6451	0.0743
Density	0.9289	1.0000	0.6451	0.0733
K-Means	0.9234	0.9990	0.6451	0.0733
LID	0.9335	0.9933	0.6451	0.0745
Hilbert	0.9329	0.9900	0.6396	0.0736

TABLE III
SciFACT RESULTS, `EF_CONSTRUCTION=32`

Strategy	Build (s)	Recall@10	NDCG@10	Retrieval (ms)
Random	0.2265	0.9980	0.6451	0.0569
Density	0.2590	0.9980	0.6451	0.0655
K-Means	0.2031	0.9967	0.6451	0.0541
LID	0.2434	0.9837	0.6420	0.0596
Hilbert	0.2127	0.9740	0.6373	0.0560

TABLE IV
FiQA RESULTS, `EF_CONSTRUCTION=200`

Strategy	Build (s)	Recall@10	NDCG@10	Retrieval (ms)
Random	28.0777	0.9992	0.3679	0.1600
Density	26.4060	0.9960	0.3662	0.1561
K-Means	21.1204	0.9975	0.3674	0.1555
LID	27.4522	0.9819	0.3646	0.1560
Hilbert	25.4640	0.9841	0.3643	0.1598

TABLE V
FiQA RESULTS, `EF_CONSTRUCTION=32`

Strategy	Build (s)	Recall@10	NDCG@10	Retrieval (ms)
Random	5.2796	0.9968	0.3617	0.1192
Density	6.1069	0.9946	0.3666	0.1335
K-Means	3.5733	0.9935	0.3632	0.1097
LID	6.0344	0.9651	0.3575	0.1278
Hilbert	4.6675	0.9517	0.3517	0.1234

TABLE VI
PREPROCESSING AND TOTAL TIME BY STRATEGY

Dataset	Strategy	Preprocess (s)	Total (s)
SciFact ef=200	Random	0.0000	0.9451
	Density	0.4569	1.3858
	K-Means	0.4448	1.3682
	LID	3.3286	4.2620
	Hilbert	3.8323	4.7652
SciFact ef=32	Random	0.0000	0.2265
	Density	0.4491	0.7081
	K-Means	0.4462	0.6492
	LID	3.3464	3.5898
	Hilbert	3.8792	4.0919
FiQA ef=200	Random	0.0000	28.0777
	Density	11.5567	37.9627
	K-Means	11.5626	32.6830
	LID	248.8068	276.2590
	Hilbert	42.5437	68.0077
FiQA ef=32	Random	0.0000	5.2796
	Density	11.5075	17.6144
	K-Means	12.4701	16.0433
	LID	250.7266	256.7609
	Hilbert	43.0635	47.7310

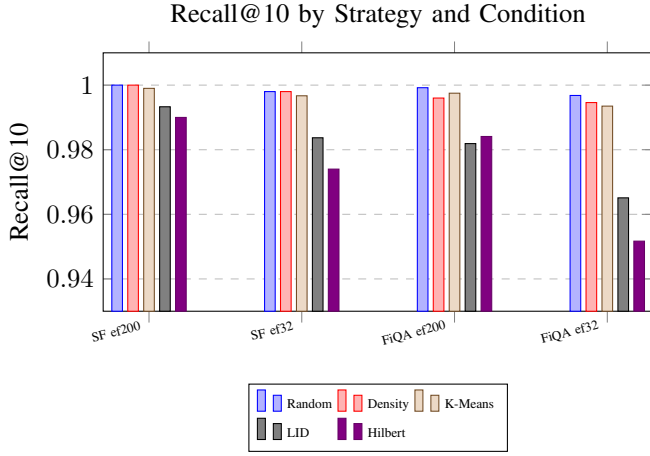


Fig. 1. Recall@10 across all five strategies and four experimental conditions. Lower ef_construction and larger corpus size amplify differences between strategies.

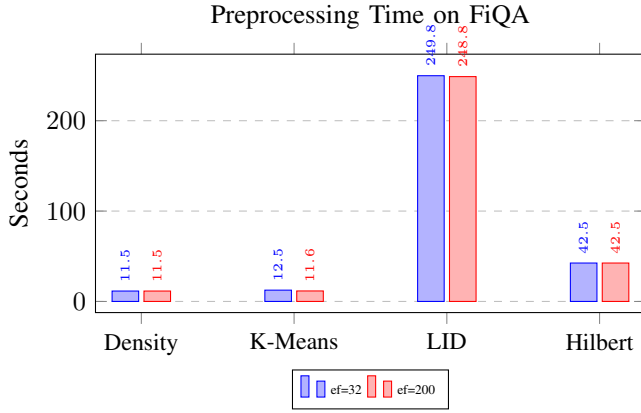


Fig. 2. Preprocessing time on FiQA by strategy. LID requires ~ 250 seconds due to $O(n^2)$ brute force neighbor computation. Density approximation achieves comparable recall at ~ 11 seconds.

NDCG@10 showed no meaningful difference across strategies within any given condition. Where differences existed they were at most 0.0055 on SciFact and 0.0162 on FiQA, suggesting that insertion order has negligible impact on downstream retrieval quality as measured against human relevance judgments.

Average retrieval time per query was effectively identical across all five strategies in every condition, with differences falling below 0.025ms. Build time differences were similarly minimal, with no strategy consistently faster or slower than the others across conditions.

The most notable Recall@10 effect was observed on FiQA with ef_construction=32, where the gap between Random (0.9968) and Hilbert (0.9517) reached 0.0451 — the largest difference observed across all conditions. LID also underperformed significantly in this condition (0.9651), falling below both K-Means and the density approximation despite being theoretically motivated by the same geometric principles.

V. DISCUSSION

The results across all conditions reveal a more nuanced picture than the initial three-strategy experiment. The density approximation matching random on SciFact is the most significant new finding and changes the framing from “random wins” to “outlier-first insertion is competitive when implemented efficiently.”

HNSW graph quality depends on early insertions forming diverse, long-range connections across the vector space. These bridge connections are most likely to be formed by high LID vectors — points that sit between clusters or on cluster boundaries, with neighbors spread across many directions. Random ordering naturally distributes high and low LID vectors throughout the insertion sequence, ensuring some high LID vectors are inserted early and form these critical connections.

K-Means clustering ordering inserts centroids first, which are low LID by definition since they represent the average position of a cluster and sit at the center of dense local neighborhoods. This delays the insertion of high LID boundary vectors until the graph is already locally saturated, reducing their ability to form the cross-cluster bridges that improve global navigability. Hilbert curve ordering groups spatially proximate vectors together, similarly concentrating low LID cluster-interior points early in the sequence. While this improves temporal cache locality during construction, it produces the same negative effect on graph connectivity, explaining why Hilbert consistently produced the lowest Recall@10 across all conditions.

The density approximation succeeds where K-Means fails precisely because it inverts the insertion priority. Rather than inserting cluster centers first, it inserts the vectors farthest from any centroid first — the geometric outliers that are most likely to be high LID. By sorting on minimum centroid distance descending, the strategy effectively approximates the outlier-first ordering that LID captures directly, without the $O(n^2)$ cost of computing exact LID. The result on SciFact — matching random at both ef_construction values — validates this geometric intuition and suggests that minimum centroid distance is a meaningful proxy for outlieriness at this corpus scale.

The underperformance of the LID implementation relative to expectations from Elliott et al. [2] is attributable to several differences between the experimental setups. Their study used seventeen embedding models across larger and more varied corpora, with a fixed ef_construction of 128 that differs from the values tested here. Their SciFact corpus contained $\sim 25,000$ documents compared to the $\sim 5,000$ document split used in this study, which yields more stable LID estimates — a larger corpus provides more neighborhood density information per vector when computing the MLE estimate. The LID implementation here is also a naive application of the MLE formula without the optimizations that may be present in their released embeddings and precomputed LID values. These differences are sufficient to explain the gap between their reported

12 percentage point improvement and the underperformance observed here.

The amplification of recall differences under FiQA with $ef_construction=32$ is consistent with the LID framework. A lower $ef_construction$ reduces the candidate pool considered when forming edges during insertion, meaning the algorithm relies more heavily on the existing graph structure to find good neighbors. When that structure is suboptimal due to poor insertion ordering, the effect compounds across a larger corpus. At $ef_construction=200$, the larger candidate pool partially compensates for suboptimal ordering, compressing the differences between strategies.

NDCG@10 remained flat across all strategies, which is expected. Insertion order affects how vectors are wired together in the graph, not the semantic content of the vectors themselves. Since NDCG@10 measures relevance against human judgments rather than mathematical nearest neighbor accuracy, it is insensitive to the graph structure differences that insertion order produces. The flat NDCG@10 results serve as a useful validation — they confirm that the recall differences observed are attributable to graph structure quality rather than any systematic bias in which documents are retrieved.

Retrieval time showed no meaningful difference across strategies, which is also expected. Query traversal time is determined primarily by ef_search and corpus size, both of which were held constant. The insertion order affects graph structure but not the traversal mechanics during search.

The preprocessing cost of each strategy is an important practical consideration. Random ordering requires no preprocessing. The density approximation and K-Means require moderate preprocessing scaling with corpus size — approximately 11–12 seconds on FiQA. LID requires ~ 250 seconds on FiQA due to the $O(n^2)$ brute force neighbor computation, and Hilbert curve fitting requires ~ 43 seconds due to the high-dimensional curve mapping. The density approximation achieves recall comparable to random and superior to LID at a fraction of the computational cost, making it the most practically favorable structured alternative tested.

VI. LIMITATIONS AND FUTURE WORK

Several limitations of this study warrant consideration when interpreting the results. First, the experiment was conducted on two datasets of moderate size — SciFact ($\sim 5,000$ documents) and FiQA ($\sim 57,000$ documents). The effects of insertion ordering may be more pronounced at significantly larger scales, such as millions of vectors, where graph connectivity decisions have longer lasting consequences and $ef_construction$ cannot feasibly be scaled to compensate.

Second, the Hilbert curve ordering was applied directly in the full 384-dimensional embedding space. The locality-preserving properties of space-filling curves degrade in high dimensions due to the curse of dimensionality, where the contrast between near and far vectors diminishes. A dimensionality reduction step such as PCA prior to curve fitting may produce a more effective ordering, though this introduces an

additional confounding variable. This tradeoff was acknowledged and accepted as out of scope for this study.

Third, hardware cache behavior was not empirically measured. Based on the properties of each strategy, Hilbert curve ordering is expected to produce the fewest cache misses during construction due to its temporal locality, while random ordering is expected to produce the most. However, the results suggest that cache efficiency during construction and graph quality are in tension — the ordering that minimizes cache misses produced the lowest recall. Empirically verifying this relationship using hardware performance counters such as Linux perf was not feasible in the current environment, as such tools require kernel-level permissions not available on the development machine.

Fourth, the LID implementation is a naive application of the MLE formula computed from 100 nearest neighbors, matching the neighbor count from Elliott et al. [2]. Future work should investigate whether a more optimized LID computation — using their released precomputed estimates or a more stable estimator — produces results closer to their reported findings at comparable corpus sizes and $ef_construction$ values.

Fifth, the density approximation was evaluated on the same two datasets as the other strategies. Its strong performance on SciFact warrants further validation at larger corpus scales where the quality of the centroid coverage may degrade. Additionally, the relationship between the number of centroids k and approximation quality is unexplored — a finer-grained centroid structure may improve outlier identification, particularly on larger and more diverse corpora.

Sixth, the machine learning approach to insertion ordering originally proposed for this study — training a model to predict optimal insertion order from dataset characteristics and observed performance metrics — was descope due to time constraints and the requirement for multiple datasets as training examples. This remains an open direction for future work, particularly as a meta-learning problem where dataset statistics serve as features for predicting which insertion strategy will perform best on unseen corpora.

Finally, NDCG@10 differences across strategies were negligible in this study. Future work on larger datasets or with more challenging query sets may reveal downstream retrieval quality differences that were not detectable at the scales tested here.

VII. CONCLUSION

This study investigated the effect of insertion ordering strategies on HNSW graph quality, comparing random ordering against four structured alternatives — K-Means clustering, Hilbert curve ordering, LID-based ordering, and a K-Means density approximation — across two datasets and two $ef_construction$ configurations. Random ordering produced the highest or joint-highest Recall@10 across most conditions, with structured orderings largely failing to improve upon the default baseline.

The exception is the density approximation, which matched random ordering on SciFact at both $ef_construction$ values.

This result validates the core geometric intuition — inserting outlier vectors first establishes the long-range bridge connections that HNSW depends on for global navigability. By approximating outlierness through minimum centroid distance rather than exact LID computation, the strategy avoids the $O(n^2)$ cost of the LID approach while preserving the underlying geometric principle.

These findings are consistent with the theoretical framework established by Elliott et al. [2], where graph quality is linked to the early insertion of high Local Intrinsic Dimensionality vectors that form critical long-range connections across the graph. Random ordering distributes these vectors naturally throughout the insertion sequence, while K-Means and Hilbert curve ordering inadvertently delay their insertion by prioritizing spatially clustered or locally proximate vectors early in the sequence. The naive LID implementation underperformed relative to their findings, attributable to differences in corpus size, number of models evaluated, ef_construction, and implementation maturity.

The practical implication is that at moderate corpus scales, the default random insertion order remains competitive. However, the density approximation offers a structured, geometrically motivated alternative that matches random performance on smaller datasets at a preprocessing cost that scales far better than exact LID computation. Future work should validate this approximation at larger scales and investigate whether a refined centroid structure further improves outlier identification.

REFERENCES

- [1] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–837, 2018.
- [2] O. Elliott *et al.*, “The impacts of data, ordering, and intrinsic dimensionality on recall in hierarchical navigable small worlds,” in *Proceedings of the 2024 ACM SIGIR International Conference on Theory of Information Retrieval*, 2024.
- [3] N. Thakur *et al.*, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.